JAVA COURSE

{ javac e package }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.0

INDICE

01 Compilare più file

02 I package

03 Organizzare un progetto

01
COMPILARE PIÙ FILE



UN FILE PER CLASSE

Con le classi il programma non sta più in un file solo: ogni classe **pubblica** ha il suo file, con lo stesso nome

- `Studente.java` *contiene la classe* `Studente`
- *Il nome deve coincidere* ***esattamente****, maiuscole comprese*
- *Un progetto reale ha centinaia di file* `.java`

```java
// Studente.java
public class Studente {
    String nome;
    int eta;
}

// Main.java
public class Main {
    public static void main(String[] args) {
        Studente s = new Studente();
        s.nome = "Marco";
        System.out.println(s.nome);
    }
}
```

COMPILARE PIÙ FILE

A `javac` si passano i file da compilare, uno per uno oppure tutti insieme con `*.java`

- *Ogni file* `.java` *produce un file* `.class`
- *Il* `.class` *contiene il* ***bytecode****, il codice che la JVM esegue*

Terminale

```
$ javac Main.java Studente.java
$ javac *.java
$ ls
Main.class      Studente.class
Main.java       Studente.java
```

ESEGUIRE IL PROGRAMMA

Si esegue la classe che contiene il `main`, scritta **senza** estensione

- `javac` *vuole il nome del* ***file****:* `Main.java`
- `java` *vuole il nome della* ***classe****:* `Main`
- *Con* `Main.class` *la JVM cerca una classe che non esiste*

Terminale

```
$ java Main
Marco
$ java Main.class
Error: Could not find or load
main class Main.class
```

LE DIPENDENZE

`Main` usa `Studente`: compilando solo `Main.java`, `javac` cerca e compila anche `Studente.java`

- *Il file deve essere dove* `javac` *lo cerca: qui nella stessa cartella*
- *Se non lo trova, la compilazione si ferma con un errore*

Terminale

```
$ javac Main.java
$ ls *.class
Main.class     Studente.class
```

CANNOT FIND SYMBOL

L'errore più comune con più file: il compilatore non trova una classe

- `Studente.java` *manca o è in un'altra cartella*
- *Il nome è scritto in modo diverso:* `studente` *non è* `Studente`

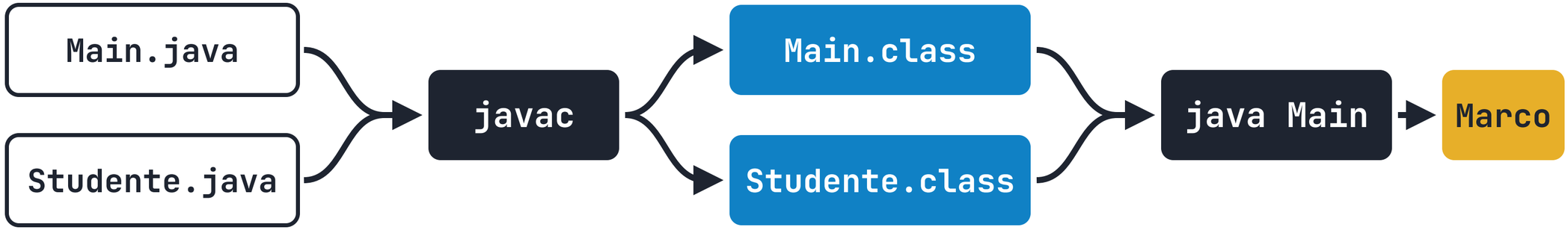
Terminale

```
$ javac Main.java
Main.java:3: error: cannot find symbol
        Studente s = new Studente();
        ^
  symbol:   class Studente
  location: class Main
```

DA .JAVA A .CLASS

Il percorso è sempre lo stesso: scrivere, compilare, eseguire

- `javac` *traduce i sorgenti in bytecode*
- `java` *avvia la JVM, che esegue il bytecode*



ESEGUIRE UN SORGENTE

Dalla versione 22 `java` esegue anche i sorgenti: compila in memoria e avvia il programma, anche con più file

- *Non crea nessun file* `.class`
- *Comodo per provare programmi piccoli*
- *Per un progetto vero si compila con* `javac`

Terminale

```
$ java Main.java
Marco
$ ls
Main.java     Studente.java
```

SORGENTI E COMPILATI

L'opzione `-d` dice a `javac` in quale cartella scrivere i `.class`

- `-d out` *crea la cartella* `out` *se non esiste*
- *I sorgenti restano puliti, i compilati stanno a parte*
- `out` *si può cancellare e ricreare quando si vuole*

Terminale

```
$ javac -d out *.java
$ ls out
Main.class     Studente.class
```

IL CLASSPATH

Il **classpath** dice alla JVM dove cercare i `.class`. Se non si indica, cerca nella cartella corrente

- *I* `.class` *sono in* `out`*: da qui la JVM non li vede*
- `-cp out` *indica la cartella giusta*
- `-cp` *è la forma breve di* `-classpath`

Terminale

```
$ java Main
Error: Could not find or load
main class Main
$ java -cp out Main
Marco
```

QUIZ

La classe `Prodotto` è scritta nel file `prodotto.java`. Il progetto compila?

```
// prodotto.java
public class Prodotto {
    String nome;
    double prezzo;
}
```

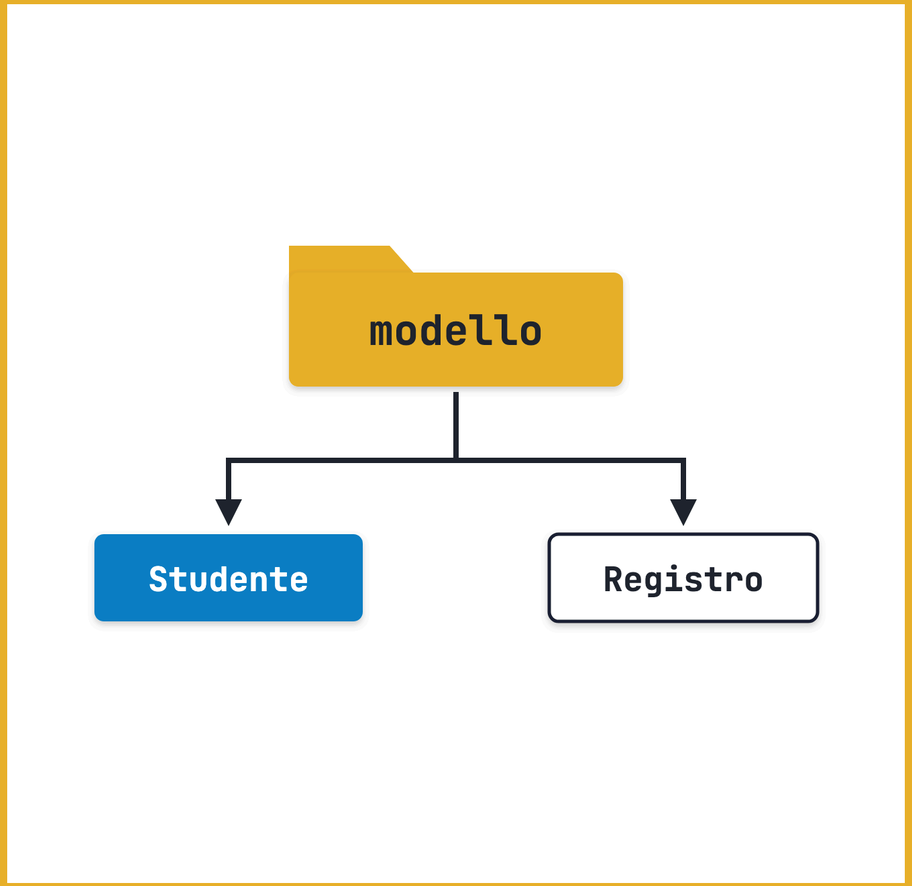
RISPOSTE

- ***No:*** *il file si chiama* `prodotto.java`*, la classe* `Prodotto`
- ***Correzione:*** *rinominare il file in* `Prodotto.java`

Terminale

```
$ javac prodotto.java
prodotto.java:1: error: class Prodotto is public, should be
declared in a file named Prodotto.java
public class Prodotto {
       ^
```

02
I PACKAGE



COS'È UN PACKAGE

Con tante classi serve ordine. Un **package** raggruppa classi che lavorano insieme e corrisponde a una **cartella**

- *Divide il progetto in parti con un ruolo chiaro*
- *Due classi con lo stesso nome possono convivere in package diversi*

```
src/
  modello/
    Studente.java
    Registro.java
  Main.java
```

DICHIARARE UN PACKAGE

La riga `package` è la **prima** del file, prima degli `import` e della classe

- *Il file sta in* `src/modello/`
- *Quindi dichiara* `package modello;`

```
package modello;

public class Studente {
    private String nome;
    private int eta;

    public Studente(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }
}
```

PACKAGE E CARTELLE

Il nome del package ricalca il percorso delle cartelle: ogni punto è una sottocartella

- `it.infobasic.registro` *diventa* `it/infobasic/registro/`
- `javac` *cerca le classi seguendo le cartelle: se non coincidono, non le trova*

```
src/
  it/
    infobasic/
      registro/
        Main.java
        modello/
          Studente.java
```

IMPORTARE UNA CLASSE

Per usare una classe di un altro package la si **importa** con `import`

- `import modello.*;` *importa tutte le classi del package*
- *Le classi dello stesso package si vedono senza* `import`
- `String`, `System` *e* `Math` *sono in* `java.lang`, *importato sempre*

```
import modello.Studente;
import modello.Registro;

public class Main {
    public static void main(String[] args) {
        Registro r = new Registro(5);
        r.aggiungi(new Studente("Marco", 20));
        r.stampaElenco();
    }
}
```

IL PACKAGE DI DEFAULT

Un file senza la riga `package` finisce nel **package di default**, che non ha nome

- *È quello usato in tutte le lezioni fino a oggi*
- *Le sue classi non si possono importare da un altro package*
- *Va bene per gli esercizi, non per un progetto*

```
// nessuna riga package
public class Main {
    public static void main(String[] args) {
        System.out.println("Ciao!");
    }
}
```

CONVENZIONE DEI NOMI

Per convenzione il nome comincia con il **dominio internet al contrario** di chi scrive il codice

- *Il dominio è unico, quindi anche il nome del package*
- *Tutto minuscolo, senza trattini né spazi*
- *I package di Java iniziano con* `java.` *e* `javax.`

```
// infobasic.it → it.infobasic
package it.infobasic.registro;
package it.infobasic.registro.modello;

// la libreria standard
import java.util.Scanner;
```

COMPILARE CON I PACKAGE

Si compila dalla radice del progetto: `-d out` ricrea dentro `out` le cartelle dei package

- *Il classpath* `-cp out` *punta alla radice dei* `.class`*, non alla cartella del package*

Terminale

```
$ javac -d out src/modello/Studente.java src/modello/Registro.java src/Main.java
$ find out -name "*.class"
out/Main.class
out/modello/Registro.class
out/modello/Studente.class
$ java -cp out Main
```

IL NOME COMPLETO

Se `Main` è in un package, per eseguirlo serve il **nome completo**: package più nome della classe

- *Si scrive con i punti, come nella riga* `package`
- *Il nome breve* `Main` *non basta*

Terminale

```
$ java -cp out it.infobasic.registro.Main
Ciao dal package
$ java -cp out Main
Error: Could not find or load
main class Main
```

QUIZ

Il file `src/util/Calcolatrice.java` contiene questo codice. Qual è il problema?

```
package strumenti;

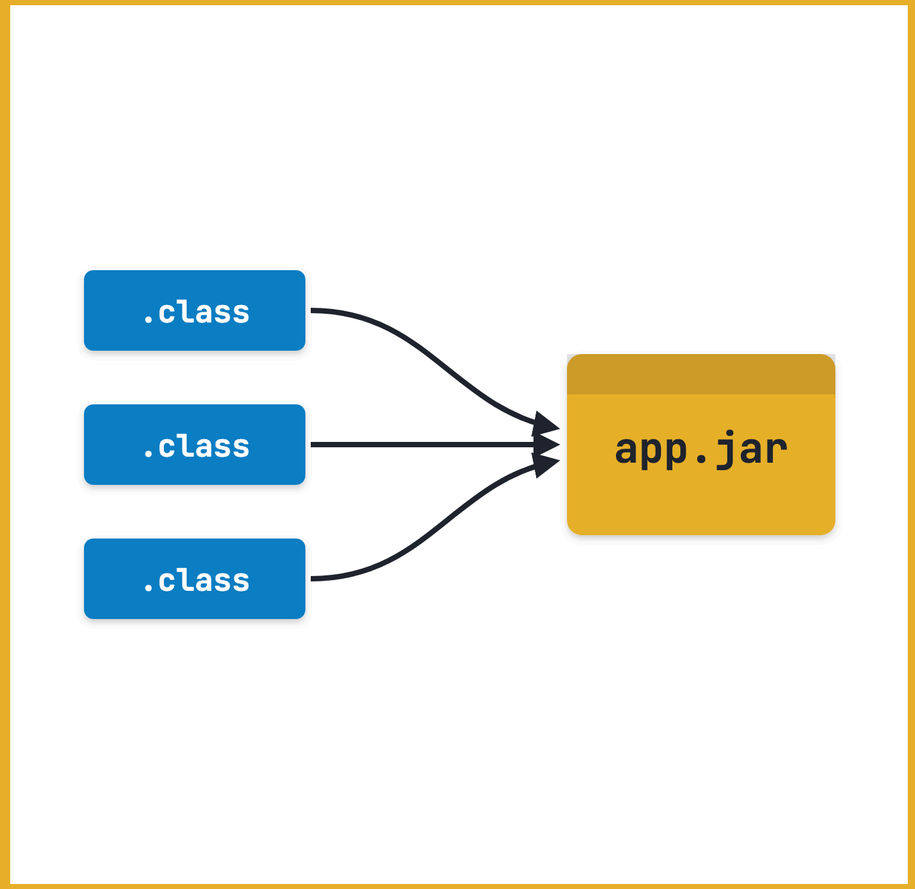
public class Calcolatrice {
    public int somma(int a, int b) {
        return a + b;
    }
}
```

RISPOSTE

- ***Problema:*** *il file è nella cartella* `util`*, il package dichiarato è* `strumenti`
- ***Conseguenza:*** `javac` *cerca* `strumenti.Calcolatrice` *in* `strumenti/` *e non la trova*
- ***Prima correzione:*** *spostare il file in* `src/strumenti/`
- ***Seconda correzione:*** *dichiarare* `package util;`

03

ORGANIZZARE UN PROGETTO



STRUTTURA STANDARD

Un progetto ordinato tiene separati sorgenti e compilati, con la stessa struttura di package

- `src` *contiene solo i file* `.java`
- `out` *contiene i* `.class` *generati da* `javac`
- `out` *non si versiona: va nel* `.gitignore`

```
progetto/
  src/
    modello/
      Studente.java
      Registro.java
    Main.java
  out/
    modello/
      Studente.class
      Registro.class
    Main.class
```

CREARE UN JAR

Un **JAR** (Java ARchive) raccoglie tutti i `.class` in un solo file, pronto da distribuire

- *È un archivio ZIP:* `-C out .` *ci mette tutto il contenuto di* `out`
- *Con* `--main-class` *si avvia con* `java -jar`

Terminale

```
$ jar --create --file app.jar --main-class Main -C out .
$ java -jar app.jar
=== Registro (3 studenti) ===
  1. Marco Rossi (20 anni)
```

DA JAVAC AI BUILD TOOL

Nei progetti reali i comandi non si scrivono a mano: li lancia un **build tool**

- *Sapere cosa fanno* `javac` *e* `java` *aiuta a capire gli strumenti*

Strumento	Cosa fa
`javac`	Compila i sorgenti, come in questa lezione
Maven	Scarica le librerie, compila, esegue i test, crea il JAR
Gradle	Fa lo stesso lavoro di Maven, usato anche per Android
IntelliJ	Lancia la compilazione da solo quando si preme Run

IL PROGETTO IN INTELLIJ

IntelliJ usa la stessa struttura: i sorgenti in `src`, i compilati in `out`

- ***Run*** *compila ed esegue, come* `javac` *più* `java`
- *Un package si crea con* ***New*** *→* ***Package*** *su* `src`
- `.idea` *e il file* `.iml` *sono la configurazione dell'IDE*

```
progetto/
  .idea/
  src/
    modello/
      Studente.java
    Main.java
  out/
    production/
  progetto.iml
```

ERRORI COMUNI

I messaggi da riconoscere quando si compila e si esegue da terminale

Messaggio	Causa
`cannot find symbol`	Classe scritta male o file non raggiungibile
`should be declared in a file named`	Nome del file diverso dal nome della classe
`package … does not exist`	Package e cartelle non coincidono
`Could not find or load main class`	Classpath sbagliato o nome della classe incompleto
`NoClassDefFoundError`	Un `.class` manca durante l'esecuzione

RIEPILOGO

- ***File:*** *una classe pubblica per file, con lo stesso nome*
- ***Comandi:*** `javac` *compila i file,* `java` *esegue la classe*
- ***Cartelle:*** `-d out` *separa i compilati,* `-cp out` *li fa trovare alla JVM*
- ***Package:*** *una cartella di classi, dichiarata nella prima riga e usata con* `import`
- ***Nome completo:*** `java -cp out it.infobasic.registro.Main`
- ***JAR:*** *tutti i* `.class` *in un file, avviato con* `java -jar`

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.0